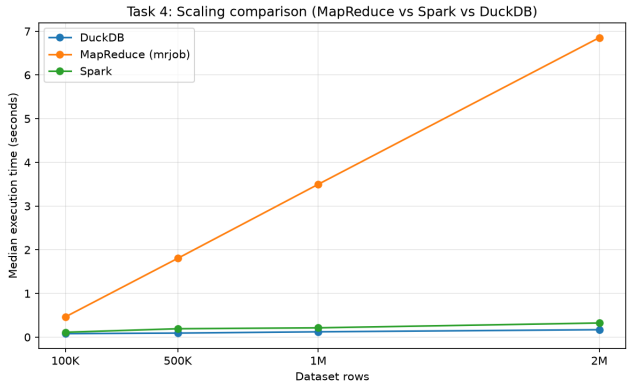
BDA Assignment 1: Observations Report

Arpita Debnath I MT25018

Scenario	Predicted best tool	Reason
aggregate 1m-row csv by pickup location	DuckDB	The data is stored in a computer's memory efficiently. The sql engine of duckdb does not have the overhead of mrjob's python on a per-row basis and the startup time of the spark jvm.
filter 2m-row parquet return 4 columns	Spark and DuckDB similar	There is support from both engines for parquet column pruning and predicate pushdown. Therefore, I expected the performance of the 4-column query to be about the same as the full column query in both the engines.
join 2m taxi rows with small zone lookup	Spark	Spark can broadcast the small lookup table instead of shuffling the large trips table. I expected this to make it as fast as or faster than duckdb.
scale aggregation 100k to 2m rows	Mapreduce grows fastest	mrjob's local runner starts a full process for every run and processes rows in plain python. so I expected its time to grow roughly in a straight line and end up the slowest of the three.
repeat same query several times	caching helps a lot	I expected spark's .cache() to give a big clear speedup over the uncached case once the cache was built.

1. Scaling Plot (Task 4)



2. Median Timing Comparison

system	task 1 (1m rows csv aggregation)	task 4 at 2m rows (csv aggregation)
Mapreduce	3.5118 s	6.9268 s
Spark	0.2487 s	0.2937 s
DuckDB	0.1181 s	0.1634 s

3. Three Observations :-

1. The greatest surprise was realized with spark. I maintained the session warm throughout runs in task 1 and task 4, instead of restarting it whenever its actual per-query time was approximately 0.25 to 0.29 seconds. It is far closer to duckdb than I had anticipated. The majority of what people claim as "spark is slow" is in reality just start-up cost of the session and jvm which had an adverse impact on the benchmarking but does not prove that the actual query is functioning slower than required.
2. The progress of mapreduce has been almost linear. Its timings increased from 0.46 seconds at 100k rows to 6.93 seconds at 2m rows. This roughly corresponds to an increase of 20 times in the rows in the table. In contrast, spark and duckdb have hardly changed their initial positions in the ranking.
3. There was a surprise in task-5. The first run was the slowest every single time. This happened even in Spark's uncached test and across five repeats of DuckDB. Since the uncached test was not caching anything, this could not be due to caching. Instead it was likely the operating system holding file bytes in memory after the first read and in combination with some warm-up time in the JVM.

4. A Prediction That Was Wrong

I had thought that Spark's broadcast join would make it comparable to Duckdb for zone lookup join which in fact turned out to be wrong. Spark did have a real broadcasthashjoin in its plan and it did make the right strategic choice. but still the average time of Duckdb of 0.0126 was about 19 times faster than that of the spark equal to 0.2390s. This case showed me that merely correctly choosing the join strategy only does save on the data shuffling phase. one has to take into account the cost of spark that it has for each query and which was more important in this case.

5. Task 1 to Task 6 Answers:-

Task-1 Q1(mapper-key value shuffle): The mapper is based on the pulocationid, as identified within each row that arrives via the filter. Once processed, the data culminates in the form of a pulocationid-fare pair moving through the shuffle system. All pairs containing a particular pulocationid are stored together in the same group so that when the reducer starts, it can access only the zone id corresponding to this pulocationid along with a list of all fare values linked to the zone identified by this id.

Task-1 Q2(hardest vs easiest to write): In terms of systems which require most clever thinking and creativity, I can name mapreduce as the most challenging in this regard. Here, I had to come up with the key-value pairs provided to the reducer and the aggregate function performed by the reducer because there's no other way of grouping it. Duckdb was the easiest by far. one group by line does the

whole job. spark landed in the middle. its dataframe code reads a lot like sql but I still had to remember that nothing actually runs until an action like count() is called.

Task-2 Q1(column filter evidence in the plans): It is easy to see the difference by examining the two saved plans. In query A's plan, all 20 columns appear under the "readschema" section. In comparison, query B's plan only lists the 4 columns requested. Both plans also display the filter applied at the scanning stage. In Spark terminology, this is termed as pushedfilters, while in DuckDB, it appears next to read_parquet. This implies that filtering is applied during the reading of data rather than afterwards.

Task-2 Q2 (why fewer columns costs less): The important point is how Parquet schemas store data. Each column is stored in a different place in the file. Therefore, by using only the 4 requested columns, the engine is able to skip reading the other 16 columns. While this does not mean that one has done less filtering, it does mean that less data has been read from disk. In DuckDB, this has resulted in real numbers, with the results from query B showing a much faster response time than query A (0.0081 vs 0.0264 seconds). Spark has also exhibited a similar pattern although it has shown a lesser difference.

Task-3 Q1(mapreduce join description): Since writing this join in mrjob is prohibited, in this way I would approach it. For each record, I would mark it with a key for the join. This is the pulocationid for trip records and the locationid for zone records. The trip record would be sent out tagged as trip with the fare attached. The zone record would be sent out tagged as zone with its borough name and zone name. The shuffle process will bring together everything with a common key. Thus, each reducer will receive all the information from one zone and all trips taking place on that pickup zone. Using a simple reduce-side join would be incorrect because it would require sending the entire multi-million row trip data through the shuffle for matching with the small lookup table. It would be better to transfer the lookup table to every mapper in advance and do the join before the shuffle process starts.

Task-3 Q2 (Spark broadcast observation): Spark's real physical plan backed this up. it showed a broadcasthashjoin with a broadcastexchange on the zone-lookup side. I didn't force this with any setting. spark's optimizer chose it on its own once it saw how small the lookup table was.

Task-4 (Curve shape): Duckdb's figures hardly change in any of the four datasets. it ranges from 0.08 sec to 0.16 sec, making it difficult to see it compared to both competitors' lines. spark too remains rather flat but is on a higher level than duckdb. this shows that most of the cost seen above is fixed because it is mostly related to the process of starting a session and planning the query rather than to the number of rows. mapreduce on the other hand is a real outlier. it rises continuously from just half a second at 100k rows to about 7 seconds at 2 million rows, meaning that the increase of 20 times in the number of rows depends on the increase in the time of the task and is simply expected due to the way mrjob is designed to operate row by row in python while using a single machine. This number only reflects mrjob running locally on one laptop. It does not tell us how a real Hadoop cluster would perform, since a cluster would split this row-by-row work across many machines instead of one.

Task-5 Q1(Do later runs get faster and why): It was noted that all conditions recorded faster times at this point too. In fact, the times dropped for the uncached Spark from 1.07 to about 0.12 to 0.16 seconds. The cache-enabled Spark followed shortly dropping its time from 0.28 seconds to approximately 0.10 to 0.11 seconds. DuckDB's time decreased from about 0.008 seconds to around 0.002 seconds. However, the interesting point is that no .cache() calls are present for the uncached case which rules out verification of the times as gain due to the use of Spark's caching. Instead, it is safe to speculate that the Operating System kept the Parquet file's bytes in RAM after the first read while also ensuring JVM terms processing of this file. So, looking at given steady-state times while comparing uncached Spark with cached Spark it is safe to say that the use of Spark's caching makes a just small benefit on top of what the OS takes care of without any major efforts.

Task-5 Q2 (When caching pays off): The 0.69-second cost of creating a cache will be worth it if you are performing multiple queries in a batch; that way, since you just pay this price once for all subsequent queries, you will get your money's worth. However, if you are running a single query and nothing more, it would not be worth waiting for the cache to be created because the overhead will not be justified.

Task 6:

Scenarios	Choice	Justification
a. 5gb parquet 16gb ram laptop 8 sql queries	Duckdb	The data can be handled without distributed systems because it is small enough to fit onto a single computer. DuckDB is able to directly read parquet with column pruning.
b. 20tb distributed immutable logs nightly fault-tolerant batch	Spark	20TB of data is larger than what just one laptop can hold. Spark gives fault tolerance for a distributed nightly job which Duckdb simply cannot do.
c. multi-stage joins plus feature transforms plus ml training on a cluster	Spark	This pipeline obtains results for various stages like features and trainings. That, in fact, is what Spark in in-memory and mllib does.
d. 30gb csv 32gb ram fast nvme 5 simple aggregates no cluster	Duckdb	All the data can already fit into the one fast machine. Since there is no cluster anyway, five simple aggregates can be handled by ordinary SQL.
e. 2gb dataset fits on one laptop spark proposed "because big data"	Duckdb	There is no need for a cluster for a 2gb dataset. To create a cluster here would be a waste of time and effort.
f. 200gb no info on format query type memory storage or distribution	Insufficient information	Without knowing the file format the query type the available memory or whether the data is already spread across machines any answer here would just be a guess.

6. Final Conclusion

Q) What factor other than dataset size most influenced your choice of system?

In my view, the most significant factor besides the size of datasets is the general costs associated with setting up the systems. DuckDB's connection costs are almost negligible this is why it had an advantage in the smallest workloads of both Task 2 and Task 3. Spark's real speed appeared after I ceased restarting its sessions before conducting any runs. At first, benchmarks were poorly designed thus showing its slower speeds than it really had. MapReduce's costs increased according to the number of rows since it is the implementation of mrjob's local runner that processes everything in plain Python, without any of the optimizations that Spark and DuckDB have in place. Ultimately, we may conclude that a new approach to measurement methods can influence the comparison of tools just as much as the workload itself influences the results.